01

PROTOCOL

CRYPTOGRAPHY ONBOARDING

HOW THE ENCRYPTION ACTUALLY WORKS

A friendly walkthrough of the cryptographic primitives, the six STARK AIR circuits, and the private subscription flow. Written for new contributors joining the team. Technical, but readable.

6	0	PQ	~9KB
STARK AIRS	TRUSTED SETUP	QUANTUM SAFE	PROOF SIZE

ONBOARDING · CRYPTOGRAPHY PRIMER

v1.0 · May 2026

Volta Team · Slashy Fx

READ THIS FIRST

Welcome. This document is the shortest path I could write between "I have heard of zero-knowledge proofs" and "I can read Protocol 01 source code and understand what each piece does." If you finish this, you can open any file in the repo and follow what is happening.

What Protocol 01 is

A privacy layer for Solana. Every transaction is exposed by default on Solana, every wallet is observable forever. Protocol 01 makes deposits, withdrawals, and recurring payments unlinkable, while staying on the same blockchain.

What this doc covers

The cryptographic primitives we use (Poseidon, Merkle, STARK), the six AIR circuits that secure every private spend, and the private subscription flow end-to-end. Skip nothing on first read.

Reading guide

1. **Page 3** — The building blocks. Hashes, commitments, nullifiers, Merkle trees. The vocabulary you need.
2. **Page 4** — What a STARK proof actually proves, in plain English, with no mathematics required.
3. **Pages 5-7** — The six AIR circuits. One page per pair of circuits, with the plain meaning and the technical layout side by side.
4. **Page 8** — The shielded pool: how shielding and unshielding work step by step.
5. **Pages 9-10** — Private subscriptions: the most interesting product surface, end to end.
6. **Page 11** — Where things live in the repo. Start contributing.
7. **Page 12** — Glossary.

Two mental models that make everything click

Model 1 — A note is a sealed envelope.

When you shield 1 SOL, you do not write "Alice deposited 1 SOL" on the blockchain. You write a sealed envelope: a hash that mathematically commits to (your secret, the amount, the epoch, the token). Anyone can see the envelope exists. Nobody can open it. Only you, holding the secret inside, can later prove "I own envelope X" without revealing X.

Model 2 — A nullifier is the envelope's death certificate.

When you spend the note, you publish a different hash derived from the same secret: the *nullifier*. The blockchain remembers every nullifier it has ever seen. If the same one shows up twice, the second attempt is rejected. The nullifier looks completely random and unconnected to the commitment, so observers cannot tell which envelope was just spent.

If you keep those two ideas in mind, the rest of the document is just "here is how we prove the envelope is real, and here is how we prove the nullifier matches it, without showing the envelope."

THE BUILDING BLOCKS

Five primitives appear everywhere in the codebase. Learn the role of each one. We will use them in every circuit on the next pages.

1. Hash function — Poseidon

A hash function turns any input into a fixed-size output that looks random. Same input always gives the same output. From the output, you cannot recover the input. SHA-256 is the famous one. We use **Poseidon** because it is "ZK-friendly": it is built from additions and multiplications inside a finite field, which a STARK circuit can express cheaply. SHA-256 inside a STARK is possible but very expensive.

```
poseidon(a, b) → one field element. Used everywhere: commitments, nullifiers, Merkle nodes.
```

Two variants in our code: Poseidon over BN254 (legacy Groth16, retired) and Poseidon over Goldilocks (every active STARK circuit). The Rust implementation lives at `stark/src/poseidon/`, the TypeScript port at `packages/zk-sdk/src/poseidon-goldilocks.ts`.

2. Commitment

A commitment is a sealed envelope. We hash together the value we want to hide and a fresh random secret. The result looks random and reveals nothing about the value. Later, anyone holding the secret can "open" the commitment by showing the value and the secret produce that exact hash. This is how a 1 SOL note can sit publicly on-chain while nobody can tell it is 1 SOL or who owns it.

```
commitment = Poseidon(nullifier_preimage, secret, deposit_epoch, token_mint)
```

3. Nullifier

A nullifier is what you publish when you spend a note. It is derived deterministically from the same secret that lives inside the commitment, but the relationship is hidden by a one-way hash. Spending a note means: prove a commitment exists for this nullifier, then write the nullifier to chain. The on-chain program stores each nullifier in a small PDA. Second attempt → PDA already exists → transaction fails. Double-spend is impossible.

```
nullifier = Poseidon(nullifier_preimage, secret)
```

4. Merkle tree

Every commitment ever made lives as a leaf in a single tree. The root of the tree is a 32-byte fingerprint of all of them. Proving "my commitment is in this tree" takes only 15 hashes (the depth) instead of looking at thousands of leaves. The tree gives us anonymity: the proof shows the leaf exists, without saying *which* leaf. Your spent note is statistically anonymous among all the other notes in the same pool.

```
root = Poseidon(Poseidon(Poseidon(leaf, sib), …), …) — depth 15
```

5. STARK proof

A STARK is a short message (~9-15 KB in our case) that mathematically convinces a verifier of a statement, without revealing the inputs that make the statement true. We prove statements like *"I know a secret whose Poseidon hash equals this commitment, and that commitment is in the tree at this root."* No trusted setup

WHAT A STARK ACTUALLY PROVES

If you have never opened a cryptography textbook, this is the page that should make the rest of the doc click. No mathematics required.

The puzzle

You have a secret value `s`. You want to convince Solana that `Poseidon(s) = some_public_commitment`, but without sending `s`. Solana then makes a decision based on your claim (release funds, activate a subscription, etc).

The trick — the "execution trace"

Computing `Poseidon(s)` is a long sequence of small steps. Each step is one round of additions and multiplications. If you write down the value of every internal variable at every step, you get a big table: rows = time, columns = state. That table is called the **execution trace**.

Inside the trace, the secret s appears in row 0. The result appears in the last row. Every step in between is a small, simple rule: *"to go from row N to row N+1, apply the Poseidon transition."*

The AIR (Algebraic Intermediate Representation)

The AIR is the precise description of those rules: "at every row, the next row must satisfy these constraints." If even one row of the trace breaks one constraint, the trace is invalid. The AIR is the same for every user: it is hard-coded in the verifier on-chain. Each Protocol 01 circuit (the next 6 pages) is one AIR.

The proof

The prover (your phone) builds the trace, commits to it with a Merkle tree, and then plays a small interactive game with the verifier where the verifier asks: "show me row 173, row 488, row 9421." If the trace was fake, almost any spot-check exposes a constraint violation. With enough random spot-checks, the verifier becomes statistically certain the trace is real. The transcript of this game compressed into one short message is the **STARK proof**.

Solana receives the proof, replays the verifier's checks deterministically, and accepts or rejects. The secret s never appeared in any message. The proof is ~9-15 KB. Verification on-chain costs ~889K compute units. No trusted setup ceremony was needed.

Why STARKs, not SNARKs (Groth16)?

Groth16 (retired)

- 256-byte proofs (great)
- Needs a trusted setup ceremony (bad)
- Built on elliptic curves — broken by Shor's quantum algorithm
- Removed from every spend path in v0.9.9

STARK (active, since v0.9.9)

- 9-15 KB proofs (larger, but uploadable in chunks)
- No trusted setup — the math alone secures it
- Hash-based (Poseidon + Blake3) — quantum-resistant
- Same prover code runs in Rust (laptop) and WASM (phone)

THE SIX AIR CIRCUITS · PART 1

Each AIR proves one statement. Each spend path combines two or three of them. Source: `stark/src/air/`. On-chain verifier: `programs/p01_stark_verifier/`.

subscriber_ownership · SIMPLEST

> IN PLAIN ENGLISH
>
> "I am the legitimate subscriber for this subscription. I know the secret that was used to create it. I am not revealing the secret."

The simplest circuit. Useful as your first read. Single Poseidon hash, 30 rounds, padded to 32 rows. Powers the on-chain claim "this caller really owns subscription X" used by pause / resume / cancel on private vaults.

Trace width	3 columns	Public input	commitment
Trace length	32 rows	Private input	subscriber_secret
Proof size	~9 KB	Statement	Poseidon(s) == commitment

1 pool_commitment · NOTE OWNERSHIP

> IN PLAIN ENGLISH
>
> "I own a denominated-pool note. The nullifier I am publishing right now was actually derived from the same secret as a real commitment in the pool."

Proves the full note-ownership chain. Three chained Poseidon hashes inside the trace: nullifier, then epoch hash, then commitment. The verifier checks that the nullifier the user is burning, and the commitment they claim to own, both came from one consistent secret + epoch + token.

Trace width	3 columns	Public inputs	nullifier, commitment
Trace length	128 rows	Private inputs	preimage, secret, deposit_epoch, token_mint
Hash cycles	3		

```
nullifier  = Poseidon(nullifier_preimage, secret)
epoch_hash = Poseidon(deposit_epoch, token_mint)
commitment = Poseidon(nullifier, epoch_hash)
```

These two circuits cover "simple ownership". Anything that needs to also prove a Merkle inclusion combines a circuit on this page with circuit 3 on the next page.

THE SIX AIR CIRCUITS · PART 2

2 balance_proof · SOLVENCY

> IN PLAIN ENGLISH
>
> "My confidential account currently has a balance, and that balance commits to the same hash the chain has stored for me. I will not say what the balance is."

Used by the confidential balance product (zkSPL). Four chained Poseidon hashes prove the commitment was built from your spending key and a balance + salt you know. The actual range check ("balance ≥ threshold") happens on-chain because the program already has the claimed balance value in its state; the AIR only proves the commitment is honest.

Trace width	4 columns	Public inputs	commitment, token_mint
Trace length	128 rows	Private inputs	balance, salt, spending_key
Hash cycles	4		

3 merkle_path · INCLUSION PROOF

> IN PLAIN ENGLISH
>
> "Here is a leaf. Here is a Merkle root. I can show, hash by hash, that the leaf really lives in the tree under that root. I will not say at which position."

The companion circuit to every spend. By itself, it proves nothing about ownership — it just proves a leaf is in the tree. Combined with circuit 1 (which proves the leaf was built from your secret), it gives the full claim "I own a real, deposited note." Canonical depth 15, so it does 15 Poseidon hashes up the tree.

Trace width	6 columns	Public inputs	leaf, root
Trace length	512 rows	Private inputs	path_elements[15], path_indices[15]
Merkle depth	15		

The trace alternates: hash the carry against the sibling, multiplex left/right by the path index, carry the result to the next level. After 15 cycles, the carry must equal the public root.

Pairing pattern: circuits 0 or 1 (ownership) + circuit 3 (inclusion) = a full anonymous spend.

THE SIX AIR CIRCUITS · PART 3

4 confidential_balance · BALANCE UPDATE

> IN PLAIN ENGLISH
>
> "My confidential account had balance B before. After this deposit or withdraw, it has balance B′. The change is consistent. I will not say what B, B′, or the amount were."

Powers the four zkSPL operations: `DEPOSIT`, `WITHDRAW`, `SEND` (private), `RECEIVE` (private). Seven chained Poseidon hashes inside the trace, plus one padding hash to make the length a power of two. The conservation law itself (*old + credit = new + debit*) is enforced by the on-chain program; the AIR proves the two commitments were built honestly from the same spending key.

Trace width	`4 columns`	Public inputs	`old_commit, new_commit, amount_hash, token_mint`
Trace length	`256 rows`	Private inputs	`spending_key, old_bal, new_bal, amount + salts`
Hash cycles	`7 + 1 padding`		

5 transfer · 2-IN 2-OUT UTXO

> IN PLAIN ENGLISH
>
> "I am consuming two notes I own, and creating two new notes. I will only show the new notes and prove the old ones are dead. The amounts balance out."

The heart of the shielded pool. Fourteen chained Poseidon hashes in a single execution trace: derive owner, derive owner-mint, build the two input commitments, derive the two nullifiers, then build the two output commitments. Each note is committed to a specific recipient via `Poseidon(recipient, mint)`, so notes are addressable but unlinkable.

Trace width	`6 columns`	Public inputs	`2 nullifiers, 2 commits, public_amount, mint`
Trace length	`512 rows`	Private inputs	`spending_key, amounts, randomness, recipients`
Hash cycles	`14 + 2 padding`		

Combined with two Merkle path proofs (one per input) at on-chain verification time, this is what lets a user split, merge, or move shielded balances without anybody on the chain seeing the link between inputs and outputs.

THE SHIELDED POOL, END TO END

Two flows. Both happen on Solana devnet right now. Both are reproducible with the mobile app.

Shield (deposit) — the easy direction

01
Generate locally
Random secret + nullifier_preimage. Compute commitment = Poseidon(preimage, secret, epoch, mint).

→

02
Send commitment
Submit shield_denominated tx with the commitment and the deposited SOL/USDC amount.

→

03
On-chain insert
zk_shielded inserts the leaf, updates the Merkle root, takes custody of the SOL in a vault PDA.

No proof is required to shield: you are giving up money, you cannot lie about that. The privacy starts later, when you spend.

Unshield (withdraw) — the interesting direction

01
Build proof locally
Phone runs Winterfell (WASM) to prove circuit 1 + circuit 3: ownership + inclusion.

→

02
Upload in chunks
Proof is ~15 KB. Split across init_proof + upload_chunk txs to p01_stark_verifier.

→

03
Verify & settle
verify_stark_proof_v2 runs on-chain; then unshield_denominated_stark releases the SOL.

What an outside observer sees

- A commitment was added to pool X at some point (anonymous).
- A nullifier was burned in pool X. The nullifier looks random.
- SOL was paid out to a recipient address.

They cannot link the burn to the deposit. The recipient address is typically a fresh stealth address, so they cannot link it to a known wallet either.

The denomination trick — why fixed-size pools

Pools are split by denomination (0.1 / 1 / 10 / 100 SOL, 1 / 10 / 100 / 1000 USDC). All notes inside one pool have the exact same value. If you shielded 1 SOL and someone watches the next withdraw of 1 SOL, they cannot deduce it was yours: every other 1-SOL note in the pool is equally likely. Mixing happens automatically. We add an *epoch maturity* delay so that timing cannot be used to correlate either.

PRIVATE SUBSCRIPTION · PART 1

Recurring payments without anybody being able to link the subscriber to the merchant. This is the most novel product in the repo.

The everyday scenario

Alice wants to subscribe to Netflix-on-Solana for 1 SOL per month. She does not want the world to know she pays Netflix, and she does not want Netflix to know her main wallet. The retailer (Netflix) wants the SOL to arrive every month, on time, with no exposure to credit card fraud or chargebacks.

The three actors

Subscriber (Alice)

Holds a secret. Already shielded 1 SOL into the denominated pool. Wants to lock it as recurring payment.

Retailer (Netflix)

A Solana pubkey. Receives the monthly payments by calling `claim_period` when due.

Subscription Vault

A PDA on-chain. Holds the funds, the rate, the interval, and the subscriber's commitment. Does not store who Alice is.

The full flow at a glance

01
Subscribe
Burn a 1-SOL note → fund a vault & bind a subscriber commitment.

02
Claim
Retailer pulls accrued periods. No subscriber action required.

03
Pause / Resume
Subscriber pauses with a STARK proof of ownership (circuit 0). Time freezes.

04
Cancel
Subscriber cancels with circuit 0. Residual SOL is re-shielded for them.

What is private, what is not

Private (cryptographically)

- Subscriber's real wallet — never on-chain
- Link between the shielded note and the vault
- Subscriber identity at pause / resume / cancel

Public (by design)

- The retailer pubkey (so they can claim)
- The rate and interval (so the contract can enforce)
- The fact that a vault exists for this retailer

The trick is on the next page: the vault is keyed by a *commitment*, not by the subscriber's pubkey. So even the retailer's own vault list looks like a pile of anonymous envelopes.

PRIVATE SUBSCRIPTION · PART 2

The four operations, in technical detail. Source: `programs/zk_shielded/src/instructions/{subscribe,claim,pause,resume,cancel}_*.rs`.

1. `subscribe_private_stark`

What: Alice burns one denominated-pool note (1 SOL) and uses the same secret to derive a fresh `subscriber_commitment`. A new `SubscriptionVault` PDA is created, keyed by (retailer, subscriber_commitment, token_mint). The 1 SOL is moved from the pool vault to the subscription vault.

Why STARK? Because burning the note requires proving circuit 1 (pool_commitment) — ownership + nullifier consistency. The instruction reads the pre-verified `ProofBuffer` from `p01_stark_verifier` and checks `verified == true` + `circuit_id == 1`.

```
vault_seeds = ["subscription_vault", retailer, subscriber_commitment, token_mint]
```

2. `claim_period`

What: The retailer calls this whenever they want to collect. The program reads the current slot, computes `claimable = floor((slot - start - paused) / interval_slots) - already_claimed`, then transfers `claimable × rate` from the vault to the retailer.

No proof needed. The retailer is the signer; this is the public side of the system. They can claim as often or as rarely as they want.

3. `pause_private_stark` / `resume_private_stark`

What: Alice pauses the subscription so it stops accruing. She is not the retailer, so she cannot sign as the vault owner. Instead, she submits a circuit 0 (subscriber_ownership) STARK proof against the vault's `subscriber_commitment`. The instruction verifies `Poseidon(subscriber_secret) == commitment` without ever seeing the secret.

Resume is the same primitive. The vault tracks `total_paused_slots` so that the time spent paused does not count toward the retailer's claimable periods.

4. `cancel_private_stark`

What: Same proof as pause (circuit 0), but the vault is closed. Any SOL the retailer had not yet claimed at the time of cancel becomes *residual* — it has to go back to Alice somehow.

The refund pipeline: The residual is routed to a `RefundJob` PDA on `p01_relayer`. A keeper later bundles an atomic transaction that shields the residual back into the denominated pool under a fresh stealth commitment Alice provided at cancel time. She receives a new shielded note, completely unlinkable to her original one.

Why this is unusual

Most "private subscription" designs in the literature handle the *send* direction (subscriber pays without revealing identity) but not the *control* direction (subscriber pauses or cancels without revealing identity). Protocol 01 uses circuit 0 specifically to give the subscriber post-subscription control rights, anchored only to a cryptographic commitment, never to a wallet. The whole life cycle is identity-free on Alice's side.

WHERE THIS LIVES IN THE REPO

A map of the source tree, ordered by “what to read first if you want to actually contribute.”

The cryptographic core

PATH	WHAT IT CONTAINS
`stark/src/air/`	The six AIR circuits in Rust (Winterfell). Each file matches one page of this doc.
`stark/src/poseidon/`	Goldilocks Poseidon implementation. Rust source of truth.
`stark/src/prover.rs`	The Winterfell prover wired to each AIR. Compiles to WASM for the phone.
`stark/src/verifier.rs`	The off-chain verifier (used in tests and dev tooling).
`packages/zk-sdk/src/poseidon-goldilocks.ts`	TypeScript port of Poseidon. Used in the mobile app.

On-chain verification

PATH	WHAT IT CONTAINS
`programs/p01_stark_verifier/`	The on-chain FRI verifier. init_proof → upload_chunk → verify. 6 circuits, ~889K compute units per verify.
`programs/zk_shielded/src/instructions/`	Shield / unshield / transfer / split / subscribe / pause / resume / cancel. All STARK-gated since v0.9.9.
`programs/p01_relayer/`	Decentralized relayer + refund pipeline. The keeper layer for cancel and bundled flows.

The subscription product

PATH	WHAT IT CONTAINS
`programs/zk_shielded/src/instructions/subscribe_private_stark.rs`	Vault creation, STARK proof buffer reading.
`programs/zk_shielded/src/instructions/claim_period.rs`	Retailer pulls. No proof.
`programs/zk_shielded/src/instructions/{pause,resume,cancel}_private_stark.rs`	Circuit 0 gated control plane.
`programs/zk_shielded/src/state/subscription_vault.rs`	The vault data structure.

Suggested first reading order

1. `stark/src/air/subscriber_ownership.rs` — smallest AIR, full Rust example.
2. `stark/src/air/merkle_path.rs` — the canonical companion circuit.
3. `programs/p01_stark_verifier/src/lib.rs` — the verifier dispatch table.
4. `programs/zk_shielded/src/instructions/subscribe_private_stark.rs` — end-to-end use of a STARK from a business-logic program.
5. `apps/mobile/services/zk/` — the client side: how a phone builds and uploads proofs.

GLOSSARY

The shortest possible definition of every term you will hit reading the code.

TERM	DEFINITION
AIR	Algebraic Intermediate Representation. The set of rules a STARK execution trace must satisfy. One circuit = one AIR.
Anchor	The Rust framework we use to write Solana programs. Provides accounts, instructions, PDAs, errors.
Blake3	A fast cryptographic hash. We use it to commit to the STARK execution trace in the Merkle hashing step.
Commitment	A hash that hides a value but binds the holder to it. Used to publish private notes on-chain.
Compute Units (CU)	Solana's gas. STARK verification on-chain costs ~889K CU; the budget cap is 1.4M.
Denomination	A fixed amount that defines a pool (0.1 / 1 / 10 / 100 SOL). All notes in one pool share it.
Devnet	Solana's development cluster. All Protocol 01 programs are deployed there today.
Epoch	In our pools, a maturity window. A note can only be unshielded once its epoch has passed, defeating timing analysis.
FRI	Fast Reed-Solomon Interactive Oracle Proof. The cryptographic engine underlying our STARK verifier.
Goldilocks	The finite field we use for STARK arithmetic: 2&sup6;^{4} − 2^{32} + 1. Fast modular reduction.
Groth16	The classic SNARK proving scheme. We retired it in v0.9.9 because of trusted setup and lack of quantum safety.
Merkle path	The list of sibling hashes you walk up the tree to prove a leaf is included under a known root.
ML-KEM-768	Post-quantum key encapsulation (FIPS 203). Used in stealth address v2 to be safe against quantum adversaries.
Nullifier	A one-time public token derived from a note's secret. Published when spending. Prevents double-spend.
PDA	Program Derived Address. A Solana account derived from seeds and a program ID; no private key, only the program can sign.
Poseidon	A hash function designed to be efficient inside ZK circuits. Every commitment / nullifier we make uses it.
Public input	A value the verifier sees as part of the proof statement. The rest of the inputs stay private.
Shielded pool	A Merkle tree of commitments. Funds enter via shield, exit via unshield. The pool is the anonymity set.
SNARK	Succinct Non-interactive Argument of Knowledge. A family of short ZK proofs. Groth16 is one. STARK is a sibling family.
STARK	Scalable Transparent Argument of Knowledge. The proof family we use today: hash-based, no trusted setup, quantum-safe.
Stealth address	A one-time recipient address derived from ECDH between sender and a receiver's viewing key. Unlinkable.
Subscription vault	The PDA holding the locked funds of a private subscription. Keyed by a commitment, not by a pubkey.